

RESUMEN TEÓRICO OBJETOS 1

1. Objetos y mensajes

Un sistema orientado a objetos es un conjunto de objetos que colaboran enviándose **mensajes**. Todo cómputo ocurre “dentro” de los objetos.

El objetivo de este enfoque es facilitar las modificaciones y extensiones sin afectar al sistema y lograr bajo acoplamiento y alta cohesión.

En un programa construido con objetos:

- No existe un “main”: la ejecución emerge del flujo de mensajes.
- Las clases se describen estáticamente, los objetos se crean dinámicamente.
- Una jerarquía de clases no indica lo mismo que la jerarquía top-down
- No hay un objeto más importante que otros. El comienzo de una aplicación depende del flujo de control, de decisiones del desarrollador, del tipo de interacción, etc.

Sistema compuesto por **objetos** que **colaboran** para llevar a cabo sus responsabilidades.

2. Responsabilidades de un objeto

Un objeto es una abstracción de una entidad del dominio del problema. Posee:

- **Identidad:**
Lo distingue de cualquier otro objeto, incluso con igual estado.
- **Estado (conocimiento):**
Representado por sus variables de instancia. Generalmente son referencias a otros objetos con los que colabora.
- **Comportamiento:**
Es el conjunto de mensajes que puede responder (su protocolo).
Define sus responsabilidades.

El objeto se define por lo que sabe hacer más que por lo que sabe almacenar.

3. Estructura interna y encapsulamiento

Los objetos poseen **estructura y comportamiento**. El comportamiento indica qué mensajes pueden entender y la estructura está especificada por variables de instancia

Un objeto se define en términos de su **comportamiento**.

- El comportamiento indica qué sabe hacer el objeto. Cuáles son sus responsabilidades.
- Se especifica a través del conjunto de mensajes que el objeto sabe responder: **protocolo**.

Las variables de instancia están encapsuladas y solo son accesibles por los métodos que ejecuta el objeto. Para acceder a una variable del objeto A, necesitamos enviarle un mensaje. Cuando un objeto recibe un **mensaje**, ejecuta el **método** que corresponde al mensaje recibido. El que envía el mensaje **delega** en el receptor la manera de resolverlo, que es privada del objeto.

Un **método** puede:

- modificar el estado interno del objeto
- colaborar con otros objetos
- retornar y terminar

Para poder enviarle un mensaje a un objeto, hay que conocerlo. Se establece una ligadura (**binding**) entre un nombre y un objeto.

Se pueden identificar tres formas de conocimiento o tipos de relaciones entre objetos.

1. Conocimiento Interno: Variables de instancia.
2. Conocimiento Externo: Parámetros.
3. Conocimiento Temporal: Variables temporales.

Además existe una cuarta forma de conocimiento especial: las pseudo-variables (como “this” o “self”).

El **encapsulamiento** es la cualidad de los objetos de ocultar los detalles de implementación y su estado interno del mundo exterior.

4. Clases

Una **clase** es una descripción abstracta de un conjunto de objetos.

Las clases cumplen tres roles:

- Agrupan el comportamiento común a sus instancias.
- Definen la estructura de sus instancias.
- Crean objetos que son instancia de ellas.

En consecuencia todas las instancias de una clase se comportan de la misma manera. Cada instancia mantendrá su propio estado interno.

Las clases se especifican por medio de un **nombre**, el **estado** o estructura interna que tendrán sus instancias y los **métodos** asociados que definen el comportamiento.

Se usa **new** para instanciar nuevos objetos. Para que un objeto esté listo para llevar a cabo sus responsabilidades hace falta **inicializarlo** (darles valor a sus variables). Esto se puede hacer mediante los constructores.

Una clase representa un concepto en el dominio del problema (o de la solución). Cuando tienen comportamiento en común hay una **Subclasificación** (subclases)

- Se reúne el comportamiento y la estructura común en una clase, la cual cumplirá el rol de **superclase**.
- Se conforma una jerarquía de clases: Luego otras clases pueden cumplir el rol de subclases, heredando ese comportamiento y estructura en común.
- Debe cumplir la relación es-un (y a veces algo más).

5. Herencia

La **herencia** es el mecanismo por el cual las subclases reutilizan el comportamiento y estructura de su superclase.

Toda relación de herencia implica:

Herencia de comportamiento

- Las subclases heredan los métodos de la superclase.
- Pueden redefinir métodos para variar comportamiento.

Herencia de estructura

- Todos los atributos se heredan.
- No se puede restringir ni renombrar atributos heredados

Lookup de métodos:

Al enviarse un mensaje a un objeto:

- 1) Se determina cuál es la clase del objeto.
- 2) Se busca el método para responder al envío del mensaje en la jerarquía, comenzando por la clase del objeto, y subiendo por las superclases hasta llegar a la clase raíz (Object)

Este proceso se denomina **method lookup**.

6. Clases abstractas

Una **clase abstracta** es una clase que no puede ser instanciada, captura comportamiento y estructura que será común a otras clases

- Se diseña sólo como clase padre de la cual derivan subclases.
- Representan conceptos o entidades abstractas.
- Sirven para factorizar el comportamiento común.
- Usualmente, tienen partes incompletas. Las subclases completan las piezas faltantes, o agregan variaciones a las partes existentes.

7. Polimorfismo

Dos o más objetos son **polimórficos** con respecto a un mensaje si: todos pueden entender ese mensaje, aún cuando cada uno lo haga de un modo diferente.

Los objetos pueden ser de distintas clases. El polimorfismo permite que haya un código más compacto.

Se consigue:

- Código genérico
- Objetos desacoplados
- Objetos intercambiables
- Objetos reutilizables
- Programar por protocolo, no por implementación

Importa el QUÉ HACER y no QUIEN LO HACE.

8. Binding dinámico

El binding dinámico es un mecanismo por el cual se decide que método responderá al mensaje recién cuando se lo recibe, en tiempo de ejecución.

Básicamente nos permite evitar el chequeo explícito de el tipo al que pertenece un objeto apuntado por una variable dejando que dicho objeto se encargue de decidir qué método se ejecuta (usando el algoritmo usual de "lookup").

Pseudo-Variables:

THIS hace referencia al objeto que lo ejecuta (al receptor del mensaje)

SUPER no puedo asignarle valor, toma valor cuando un objeto comienza a ejecutar un método. En relación a "this", solo cambia la forma en la que se hace el method lookup. La búsqueda de métodos comienza en la clase inmediata superior a aquella donde está definido el método que envía el mensaje (sin importar la clase del receptor)

- Se utiliza solamente para extender comportamiento heredado

9. Tipos e interfaces

Un **tipo** en un lenguaje orientado a objetos es un conjunto de firmas de métodos.

Una **interfaz** es similar a una clase abstracta, solo que define un tipo sin implementar comportamiento. Permite desacoplar comportamiento de su implementación concreta. Es útil cuando varias clases comparten una misma forma de uso.

- Cada clase o interfaz en Java define un tipo
- El tipo de una variable, es el que indico al declararla

En compilación

- Cada variable tiene un tipo
- En cada asignación se chequea que el tipo del valor (derecha) sea el mismo o un subtipo del de la variable (izquierda)
- En cada envío de mensaje se chequea que el tipo de la variable incluya la firma del mensaje

En ejecución

- Cada objeto es instancia de una clase (eso nunca cambia)
- Cuando el objeto recibe un mensaje, el método se comienza a buscar en su clase (binding dinámico)
- El tipo de la variable que apunta al objeto "no importa"

10. Testing

El **testing** forma parte del programa, se escribe test para encontrar errores. Estos errores pueden ser:

- El programa no hace lo que tiene que hacer.
- Hace algo mal.
- El programa "explota".

Hacer testeo de unidad en objetos es asegurarse de que el programa **hace lo que se espera**, **lo hace como se espera**, y **no falla**. Es importante testear temprano, y tanto como sea el riesgo del artefacto a testear.

Hay distintos tipos de test pero ahora nos importan **3 tipos clave**:

- **Tests funcionales:**
 - Verifican que el programa hace lo que tiene que hacer.
- **Tests de unidad:**
 - Asegura que la unidad mínima (método) de nuestro programa funciona correctamente, y aislada de otras unidades.
 - Testear un método es confirmar que el mismo acepta el rango esperado de entradas, que retorna el valor esperado en cada caso y maneja valores borde (extremos).
- **Tests automatizados**
 - Software para guiar la ejecución de los tests y controlar los resultados. Básicamente programas que testean programas.
 - Son "parte del software" (y un indicador de su calidad).

JUnit es una herramienta para simplificar la creación de los test de unidad y automatizar su ejecución y reporte. Ayuda a escribir/programar tests útiles.

Los tests que escribimos con JUnit son **tests funcionales, automatizados, de unidad**.

- Hay una clase de test por cada clase a testear
- Un método que prepara lo que necesitan los tests (con BeforeEach) y queda en variables de instancia
- Uno o varios métodos de test por cada método a testear
- Hay independencia entre test. No puedo asumir que otro test se ejecutó antes o se ejecutará después del que estoy escribiendo.

La **estrategia general** para testear es pensar qué podría variar (qué valores puede tomar) y que pueda causar un error o falla. Para eso usamos **dos estrategias**:

- Escribo tests de **particiones equivalentes** en donde identifico particiones, y elijo valores representativos dentro y fuera de cada partición para usarlos en los tests
 - Conjunto de casos que prueban lo mismo o revelan el mismo bug.
 - Si se trata de casos en un conjunto, tomo un caso que pertenezca al conjunto y uno que no.
 - Si se trata de valores en un rango, tomo un caso dentro y uno por fuera en cada lado del rango
- Escribo tests con **valores de borde** en donde identifico los bordes en las particiones de equivalencia y uso valores en esos bordes para mis tests.
 - Los errores ocurren con frecuencia en los límites y ahí es donde los vamos a buscar.
 - Intentamos identificar bordes en nuestras particiones de equivalencia y elegimos esos valores
 - Buscar los bordes en propiedades del estilo: velocidad, cantidad, posición, tamaño , duración, edad, etc.

Escribir tests de unidad (con JUnit) es parte “programar” y nos **ayuda a entender que se espera de nuestros objetos**.

11. Colecciones

Las **colecciones** tienen punteros a objetos. Si modifico un objeto, alcanza a todos los que tenía puntero. Se usan cuando las relaciones son 1 a muchos.

Todos los lenguajes OO ofrecen librerías de colecciones. Las colecciones admiten, generalmente, contenido heterogéneo en términos de clase, pero homogéneo en términos de comportamiento, es decir, en las colecciones puede haber objetos de distintas clases, pero tienen que ser polimórficos (mismo tipo), para que puedan entender el mismo mensaje.

Siempre (o casi) que tenemos colecciones repetimos las **mismas operaciones**. Un ejemplo de esto es recorrer colecciones. En JAVA todas las colecciones entienden el mensaje `iterator()` que retorna un objeto que abstrae la forma en la que se recorre la colección, sin importar la clase de colección con la que trato.

- Un Iterator encapsula:
 - Cómo recorrer una colección particular
 - El estado de un recorrido
- No nos interesa la clase del iterador (son polimórficos)
- El loop `for-each` esconde la existencia del iterador

IMPORTANTE

- Nunca modifico una colección que obtuve de otro objeto.

- Cada objeto es responsable de mantener los invariantes de sus colecciones.
- Solo el dueño de la colección puede modificarla.
- Recordar que una colección puede cambiar luego de que la obtengo

11. Streams y lambdas

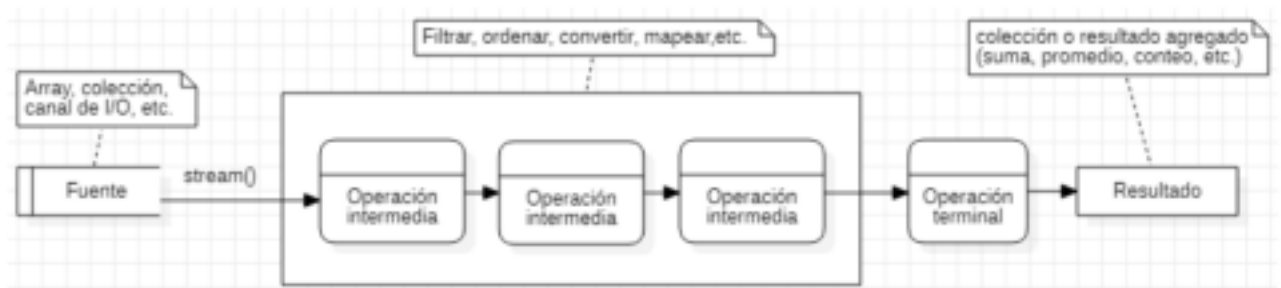
Las **Expresiones Lambda** son métodos anónimos. Los **streams** permiten procesamiento funcional de colecciones, las operaciones se combinan para formar pipelines (tuberías)

Los streams:

- No almacenan sino que proveen acceso a una fuente (colección, canal I/O, etc.).
- Cada operación produce un resultado, pero no modifica la fuente.
- Potencialmente sin final.
- Consumibles: cada elemento se visita una sola vez.

Para construir un pipeline **encadena envíos de mensajes:**

- Una fuente, de la que se obtienen los elementos
- Cero o más operaciones intermedias, que devuelven un nuevo stream
- Operaciones terminales, que retornan un resultado



Para filtrar una colección en Java, es recomendable utilizar el protocolo de streams mediante **filter()**, que solo “deja pasar” los elementos que cumplen cierto predicado.

map() nos da un stream que transforma cada elemento de entrada aplicando una función que indiquemos. La función de transformación (de mapeo) recibe un elemento del stream y devuelve un objeto

collect() es una operación terminal que nos permite obtener un objeto o colección de objetos a partir de los elementos del stream.

11. Diseño OO y Casos de uso.

Un buen Diseño OO debe cumplir con **alta cohesión y bajo acoplamiento**.

Análisis → investigación del problema y requisitos.

Diseño → solución conceptual que satisface los requisitos.

Los **casos de uso** se definen para satisfacer los objetivos de usuario o actores principales. Sirven para encontrar objetivos que requieren los actores principales.

Permiten:

- Identificar actores.
- Identificar objetivos.
- Definir interacciones externas esperadas.

El diagrama muestra:

- Actores externos.
- Casos de uso del sistema.

Los **DSS** (Representación de los Diagramas de Secuencia del Sistema) **derivan de los casos de uso**. Permite determinar qué objeto envía y recibe cada mensaje e identificar operaciones del sistema.

12. Modelo de dominio

El **modelo del dominio** permite encontrar relaciones entre las clases y determinar atributos. Es útil para representar el dominio del problema en la etapa de análisis. La tarea central es identificar las clases conceptuales relacionadas con el escenario que se está diseñando.

“Es mejor especificar en exceso un modelo del dominio con muchas clases conceptuales de grano fino que especificar por defecto en forma general”.

Consejos:

- Representar problema, no solución.
- Omitir detalles irrelevantes.
- Evitar sinónimos.
- Descubrir conceptos del mundo real.

Estrategias:

- Identificación de frases nominales: encontrar conceptos (y sus atributos) mediante la identificación de los sustantivos en la descripción textual del dominio del problema.
- Utilización de una lista de categorías de clases conceptuales:

Categoría de Clase Conceptual	Ejemplos
Objeto físico o tangible	Libro impreso
Especificación de una cosa	Especificación del producto, descripción
Lugar	Tienda
Transacción	Compra, pago, cancelación
Roles de la gente	cliente
Contenedor de cosas	Catálogo de libros, carrito, Avion
Cosas en un contenedor	Libro, Artículo, Pasajero
Otros sistemas	Sistema de facturación de AFIP
Hechos	cancelación, venta, pago
Reglas y políticas	Política de cancelación
Registros financieros/laborales	Factura/ Recibo de compra
Manuales, documentos	Reglas de cancelación, cambios de categoría de cliente

Para construir el Modelo del Dominio:

1. Listar los conceptos candidatos
2. Graficarlos en un Modelo del Dominio: Un Modelo del Dominio es una representación visual de las clases conceptuales del mundo real en un dominio de interés.
3. Agregar atributos a los conceptos: Se identifican los atributos que son necesarios para satisfacer los requerimientos de información de los casos de uso en desarrollo. Los atributos en un modelo deberían ser, preferiblemente, atributos simples o tipos de datos primitivos. (Recuerde relacionar las clases conceptuales con asociaciones, no con atributos)
4. Agregar asociaciones entre conceptos.

En **UML**, la relación de conocimiento entre objetos o instancias de clases: se modela con una asociación hacia él/los objetos que se conocen, agregando en el final de la asociación nombre (rol) y multiplicidad.

Si algo está poco tiempo en el sistema, conviene no representarlo.

El **Modelo del Dominio** debe incluir todas las clases candidatas que identifiquemos a partir de los Casos de Uso y de la Lista de Categorías, luego pueden revisarse en el Diagrama de Clases.

Los **contratos** son una de las formas de describir el comportamiento del sistema en forma detallada. Describen pre y post condiciones para las operaciones.

Las secciones de un contrato son:

- Operación: nombre de la operación y parámetros.
- Precondiciones: Suposiciones relevantes sobre el estado del sistema o de los objetos del Modelo del Dominio, antes de la ejecución de la operación.
 - No se validarán dentro de la operación, sino que se asumirán como verdaderas.
 - Son suposiciones no triviales que el lector debe saber que se hicieron.
- Postcondiciones: el estado del sistema o de los objetos del Modelo del Dominio, después de que se complete la ejecución de la operación.
 - Describen el estado y cambios del sistema después de ejecutarse la operación, utilizando conceptos del modelo conceptual o del Dominio

13. Heurísticas para Asignación de Responsabilidades (HAR)

Las Heurísticas para Asignación de Responsabilidades (HAR) promueven ambos **Alta Cohesión y Bajo Acoplamiento**.

La habilidad para asignar responsabilidades es extremadamente importante en el diseño. La asignación de responsabilidades generalmente ocurre durante la creación de diagramas de interacción.

- **Experto en Información:** es el objeto que sabe unir responsabilidades parciales para lograr un todo. Es la clase que tiene la información necesaria para realizar la responsabilidad.
- **Creador:** asignar a la clase B la responsabilidad de crear una instancia de la clase A si:
 - B contiene objetos A (agregación, composición).
 - B registra instancias de A.
 - B tiene los datos para inicializar objetos A.
 - B usa a objetos A en forma exclusiva

La intención del Creador es encontrar una clase que necesite conectarse al objeto creado en alguna situación. Eligiéndolo como el creador se favorece el bajo acoplamiento.
- **Controlador:** maneja eventos del sistema y Representa al sistema, subsistema o dispositivo. La intención del Controlador es encontrar manejadores de los eventos del sistema, sin recargar de responsabilidad a un solo objeto y manteniendo alta cohesión.

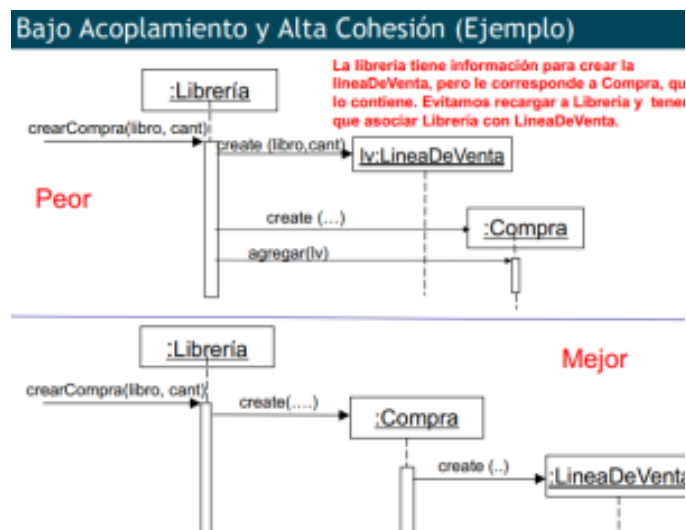
14. Bajo acoplamiento y alta cohesión.

El **Bajo Acoplamiento** es asignar responsabilidades de manera que el acoplamiento permanezca lo más bajo posible. El **acoplamiento** es una medida de dependencia de un objeto con otros. Es bajo si mantiene pocas relaciones con otros objetos.

- **Asignar responsabilidad de manera que el acoplamiento se mantenga lo más bajo posible.**
- **El alto acoplamiento dificulta el entendimiento y complica la propagación de cambios en el diseño.**

La **Alta Cohesión** es asignar responsabilidades de manera que la cohesión permanezca lo más fuerte posible. La cohesión es una medida de la fuerza con la que se relacionan las responsabilidades de un objeto, y la cantidad de ellas.

- **Ventaja:** clases más fáciles de mantener, entender y reutilizar.



Del análisis al diseño

- Los casos de uso sugieren los eventos del sistema que se muestran en los diagramas de secuencia del sistema.
- En los contratos de las operaciones, utilizando conceptos del Modelo del Dominio, se describen los efectos que dichos eventos (operaciones) producen en el sistema.
- Los eventos del sistema representan los mensajes que dan inicio a Diagramas de Secuencia del Diseño, mostrando las interacciones entre los objetos del sistema.
- Los objetos con sus métodos y relaciones se muestran en el Diagrama de Clases del Diseño (basado en el Modelo del Dominio).



Pasos

- 1) Cree un diagrama de secuencia por cada operación asociada al caso de uso.
- 2) Si el diagrama queda complejo, sepárelo en diagramas menos complejos (uno por cada escenario).
- 3) Use el contrato de la operación como punto de partida; piense en objetos que colaboran para cumplir la tarea (la mayoría de estos objetos están definidos en el modelo del dominio).
- 4) Aplique las **Heurísticas para Asignación de Responsabilidades (HAR)** para obtener un mejor diseño.

15. HAR adicionales

Polimorfismo: cuando el comportamiento varía según el tipo, asigne la responsabilidad a los tipos/las clases para las que varía el comportamiento. Nos permite sustituir objetos que tienen idéntica interfaz.

“No hables con extraños”: evite diseñar objetos que recorren largos caminos de estructura y envían mensajes (hablan) a objetos distantes o indirectos (extraños). Dentro de un método sólo pueden enviarse mensajes a objetos conocidos:

- Self/this
- un parámetro del método
- un objeto que esté asociado a self/this
- un miembro de una colección que sea atributo de self/this
- un objeto creado dentro del método

Los demás objetos son extraños (strangers)

Heurísticas para Diseño “ágil” Orientado a Objetos (Principios S O L I D):

S - SRP: The Single-Responsibility Principle

Principio de Responsabilidad única. Una clase debería cambiar por una sola razón. Debería ser responsable de únicamente una tarea, y ser modificada por una sola razón (alta cohesión).

O - OCP: The Open-Closed Principle

Entidades de software (clases, módulos, funciones, etc.) deberían ser “abiertas” para extensión, y “cerradas” para modificación.

Abierto a extensión: ser capaz de añadir nuevas funcionalidades.

Cerrado a modificación: al añadir la nueva funcionalidad no se debe cambiar el diseño existente.

L - LSP: The Liskov Substitution Principle

Los objetos de un programa deben ser intercambiables por instancias de sus subtipos sin alterar el correcto funcionamiento del programa.

Es decir, que si el programa utiliza una clase (clase A), y ésta es extendida (clases B, C, D, etc...) el programa tiene que poder utilizar cualquiera de sus subclases y seguir siendo válido. Uso correcto de herencia (Is-a) y polimorfismo.

I - ISP: The Interface-Segregation Principle

Las clases que tienen interfaces “voluminosas” son clases cuyas interfaces no son cohesivas.

Las clases no deberían verse forzadas a depender de interfaces que no utilizan. Cuando creamos interfaces (protocolos) para definir comportamientos, las clases que las implementan, no deben estar forzadas a incluir métodos que no va a utilizar.

D - DIP: The Dependency-Inversion Principle

a. Los módulos de alto nivel de abstracción no deben depender de los de bajo nivel.

b. Las abstracciones no deben depender de detalles. Los detalles deben depender de las abstracciones.

Módulos de alto nivel: se refieren a los objetos que definen qué es y qué hace el sistema.

Módulos de bajo nivel: no están directamente relacionados con la lógica de negocio del programa (no definen el dominio). Por ejemplo, el mecanismo de persistencia o el acceso a la red.

Abstracciones: se refieren a protocolos (o interfaces) o clases abstractas.

Detalles: son las implementaciones concretas, (cuál mecanismo de persistencia, etc). Ser capaz de «invertir» una dependencia es lo mismo que ser capaz de «intercambiar» una implementación concreta por otra implementación concreta cualquiera, respecto a la misma abstracción

16. Herencia vs. Composición

Herencia: Las clases y los objetos creados mediante herencia están **fuertemente acoplados** ya que cambiar algo en la superclase afecta directamente a la/las subclases.

- La herencia es total. **Caja blanca.**
- Usualmente debemos redefinir o anular métodos heredados.
- Los cambios en la superclase se propagan automáticamente a las subclase.

Composición: Las clases y los objetos creados a través de la composición están **débilmente acoplados**, lo que significa que se pueden cambiar más fácilmente los componentes sin afectar el objeto contenedor.

- Un objeto se compone de forma dinámica. Uno puede cambiar las instancias de los objetos con los que se colabora en el momento de ejecución. **Caja negra.**
- Los objetos pueden reutilizarse a través de su interfaz (sin conocer el código).
- A través de las relaciones de composición se pueden delegar responsabilidades entre los objetos.

Regla: si la herencia obliga a negar comportamiento o hereda demasiado → **usar composición.**

17. Smalltalk y JavaScript

Smalltalk

- Lenguaje OO puro: todo es un objeto.
 - Las clases también son objetos, tienen sus propios atributos y métodos. El `new`, por ejemplo, es un mensaje de clase. Las clases son instancias de una clase también (su metaclass). Por cada clase hay una metaclass (se crean juntas).
 - Está abierto a modificación. Incluso lo que comúnmente conocemos como estructuras de control (como el `if`, `while`, etc.) se implementa como envíos de mensajes a objetos.
- Es un lenguaje dinámico, en el que no se indica explícitamente el tipo de las variables.
- Propone una estrategia exploratoria (construccionista) al desarrollo de software.
- El ambiente es tan importante como el lenguaje.
 - Está implementado en Smalltalk.
 - Ricas librerías de clases (fuentes de inspiración y ejemplos).
 - Todo su código fuente está disponible y es modificable.
 - Tiene su propio compilador, debugger, editor, inspector, perfilador, etc.
 - Es extensible.
- Sintaxis minimalista (con sustento en su foco educativo).
- Fuente de inspiración de casi todo lo que vino después (en OO).

JavaScript (ECMAScript)

- Lenguaje de propósito general.
- Es un lenguaje dinámico, en el que no se indica explícitamente el tipo de las variables.
- Multiparadigma.
- Se adapta a una amplia variedad de estilos de programación.
- Pensado originalmente para scripting de páginas web.
- Con una fuerte adopción en el lado del servidor (NodeJS).
- Basado en objetos (con base en prototipos en lugar de clases).
 - En Javascript no tengo clases.
 - La forma más simple de crear un objeto es mediante la notación literal (estilo JSON).
 - Cada objeto puede tener su propio comportamiento (métodos).
 - Cada objeto hereda comportamiento y estado de su prototipo.
 - Cualquier objeto puede servir como prototipo de otro.
 - Puedo cambiar el prototipo de un objeto (y así su comportamiento y estado).
 - Termina formando cadenas de delegación.
 - Se puede asignar funciones como constructores.